

Практическая работа №1

Дисциплина: Математика в программировании

Тема: Предобработка данных интеллектуальных моделей

Предметная область: медицинская диагностика сердечно-сосудистых заболеваний

Набор данных: UCI Heart Disease

Преподаватель: Холмогоров В. В.

Москва, 2026 г.

Содержание

Введение

1. Теоретическая часть
 - 1.1. Описание набора данных
 - 1.2. Постановка задачи классификации
 - 1.3. Методы предобработки данных
 - 1.4. Аугментация табличных данных
 - 1.5. Методы визуализации и понижения размерности
 - 1.6. Метрики качества классификации
 2. Практическая часть
 - 2.1. Основные компоненты программной реализации
 - 2.2. Загрузка и первичный анализ данных
 - 2.3. Реализованная предобработка
 - 2.4. Проверка пригодности данных на модели
 - 2.5. Аугментация данных
 - 2.6. Визуализация данных
- Заключение
- Список использованных источников
- Приложение А

Введение

Цель работы - выполнить полный цикл предварительной обработки табличного медицинского набора данных для последующего решения задачи бинарной классификации наличия сердечно-сосудистого заболевания у пациента.

В качестве прикладной задачи выбрана диагностика признаков болезни сердца по клиническим характеристикам пациента. Исходная целевая переменная набора UCI Heart Disease принимает значения от 0 до 4, где 0

означает отсутствие заболевания, а значения 1-4 соответствуют наличию заболевания различной выраженности. В данной работе эта переменная преобразована в бинарную: 0 - заболевание отсутствует, 1 - заболевание присутствует.

Актуальность работы обусловлена двумя причинами. Во-первых, сердечно-сосудистые заболевания остаются одной из наиболее значимых медицинских проблем: по данным Всемирной организации здравоохранения, они являются ведущей причиной смерти в мире, а раннее выявление риска важно для начала лечения и профилактики [1]. Во-вторых, с учебной точки зрения работа направлена на освоение базового цикла подготовки данных: анализа качества, обработки пропусков, кодирования категориальных признаков, стандартизации, аугментации и визуального анализа.

Для решения задачи используются методы Python: `pandas` и `NumPy` для обработки таблиц, `seaborn` и `matplotlib` для визуализации, `scikit-learn` для построения конвейера предобработки, стандартизации, `one-hot encoding`, PCA, t-SNE и проверки результата на логистической регрессии. Выбор этих инструментов обусловлен тем, что они являются стандартными средствами анализа табличных данных и машинного обучения в Python [3-8].

Работа состоит из введения, теоретической части, практической части, заключения, списка источников и приложения с основным программным кодом. В теоретической части описаны набор данных, термины и методы предобработки. В практической части приведены результаты загрузки, анализа качества данных, предобработки, аугментации, визуализации и проверки подготовленной матрицы признаков на простой модели.

1. Теоретическая часть

1.1. Описание набора данных

В работе используется набор данных **UCI Heart Disease**, размещённый в репозитории машинного обучения UCI [2]. Набор относится к области Health and Medicine и предназначен для задач классификации. В исходном описании указано, что база содержит данные из четырёх медицинских учреждений: Cleveland Clinic Foundation, Hungarian Institute of Cardiology, V.A. Medical Center Long Beach и University Hospital Zurich.

Полная версия содержит 76 атрибутов, однако в опубликованных экспериментах обычно применяется подмножество из 14 основных признаков [2]. В данной работе используются файлы `processed.cleveland.data`, `processed.hungarian.data`,

processed.switzerland.data и processed.va.data, так как они приведены к общей 14-признаковой схеме.

Параметр	Значение
Предметная область	Медицинская диагностика сердечно-сосудистых заболеваний
Тип данных	Табличные медицинские данные
Количество строк после объединения	920
Количество строк после удаления дубликатов	918
Количество исходных признаков	13 входных признаков и целевая переменная num
Тип задачи	Бинарная классификация
Целевая переменная	target: 0 - болезни нет, 1 - болезнь есть

Используемые признаки:

Признак	Описание
age	возраст
sex	пол
cp	тип боли в груди
trestbps	давление в покое
chol	холестерин
fbs	сахар натощак выше 120 mg/dl
restecg	результат ЭКГ в покое
thalach	максимальная достигнутая частота сердечных сокращений
exang	стенокардия при нагрузке
oldpeak	депрессия ST
slope	наклон сегмента ST
ca	число крупных сосудов
thal	результат thal-теста
num	исходная целевая переменная

1.2. Постановка задачи классификации

Классификация - это задача машинного обучения, в которой объекту требуется назначить один из заранее известных классов. В данной работе решается бинарная классификация, поскольку классов два: отсутствие заболевания и наличие заболевания.

Исходная переменная `num` содержит значения 0, 1, 2, 3, 4. Для диагностической постановки она преобразуется следующим образом:

```
target = 0, если num = 0  
target = 1, если num > 0
```

Такой подход соответствует описанию UCI: в экспериментах с этим набором часто различают отсутствие заболевания, обозначенное значением 0, и наличие заболевания, обозначенное значениями 1-4 [2].

1.3. Методы предобработки данных

Предобработка данных - это набор действий, направленных на приведение исходного набора к виду, пригодному для анализа и обучения модели. В работе применяются очистка данных, обработка пропусков, удаление признака с большой долей пропусков, создание индикаторов пропусков, стандартизация и кодирование категориальных признаков.

1.3.1. Очистка данных

Дубликат - это строка, полностью совпадающая с другой строкой набора данных. Дубликаты могут исказить статистику и завышать влияние отдельных наблюдений. В работе дубликаты удаляются методом `drop_duplicates()`.

Пропуск - это отсутствующее значение признака. В исходных файлах UCI пропуски обозначены символом вопроса. При загрузке данных этот символ преобразуется в `NaN`. Также в работе выявлены скрытые пропуски: значения `chol = 0` и `trestbps = 0`, которые медицински неправдоподобны и поэтому заменяются на `NaN`.

1.3.2. Импутация пропущенных значений

Импутация - это заполнение пропущенных значений. В `scikit-learn` для этого используется `SimpleImputer`, который заменяет пропуски статистикой по столбцу, например средним, медианой или самым частым значением [3].

Для числовых признаков используется медиана. Медиана - это значение, которое делит упорядоченный набор пополам. Она менее чувствительна к

выбросам, чем среднее арифметическое. Для категориальных признаков используется мода, то есть наиболее часто встречающаяся категория.

1.3.3. Стандартизация

Стандартизация приводит числовые признаки к сопоставимому масштабу. В scikit-learn `StandardScaler` рассчитывает стандартизированное значение по формуле [4]:

$$z = (x - u) / s$$

где x - исходное значение признака, u - среднее значение признака на обучающих данных, s - стандартное отклонение признака, z - стандартизированное значение.

Стандартное отклонение показывает типичный разброс значений относительно среднего. Для генеральной совокупности оно вычисляется по формуле:

$$\sigma = \sqrt{(1 / N) \cdot \sum (x_i - \mu)^2}$$

где σ - стандартное отклонение, N - число объектов, x_i - отдельное значение, μ - среднее значение, Σ - знак суммирования.

1.3.4. One-hot encoding

One-hot encoding - это способ преобразования категориального признака в набор бинарных столбцов. Например, признак `ср` с четырьмя типами боли в груди преобразуется в несколько столбцов вида `ср_1.0`, `ср_2.0`, `ср_3.0`, `ср_4.0`. В scikit-learn для этого используется `OneHotEncoder` [5].

Этот метод нужен потому, что коды категорий нельзя трактовать как обычные числа. Если оставить категорию в виде числа, модель может ошибочно считать, что категория 4 больше категории 2 в количественном смысле.

1.3.5. Pipeline и ColumnTransformer

`Pipeline` - это последовательность преобразований, выполняемых над данными. `ColumnTransformer` позволяет применять разные преобразования к разным группам столбцов. В данной работе числовые признаки проходят импутацию и стандартизацию, категориальные - импутацию и one-hot encoding, а индикаторы пропусков передаются без изменений.

1.4. Аугментация табличных данных

Аугментация данных - это искусственное увеличение набора данных за счёт создания новых объектов на основе существующих. Для табличных данных нельзя использовать методы, характерные для изображений, поэтому применена статистическая аугментация.

В работе используется сэмплирование меньшего класса с возвращением и добавление малого гауссова шума к числовым признакам. Сэмплирование - это случайный выбор строк из набора данных. Сэмплирование с возвращением означает, что одна и та же строка может быть выбрана несколько раз. Это позволяет создать нужное число базовых заготовок для синтетических объектов.

Гауссов шум - это случайное отклонение, взятое из нормального распределения. В работе масштаб шума равен 3% от стандартного отклонения признака внутри класса:

```
noise_scale = 0.03 * std
```

где `std` - стандартное отклонение признака. Такой шум делает новые записи немного отличающимися от исходных, но сохраняет их правдоподобность.

После добавления шума значения ограничиваются 1% и 99% квантилями. Квантиль - это значение, ниже которого находится заданная доля наблюдений. Ограничение нужно, чтобы искусственные значения не уходили в нереалистичные медицинские диапазоны.

1.5. Методы визуализации и понижения размерности

Визуализация данных применяется для разведочного анализа: она помогает увидеть распределения признаков, пересечение классов, выбросы и возможную структуру данных.

Гистограмма показывает распределение одного числового признака по интервалам. В работе она используется для анализа возраста пациентов по классам. Диаграмма рассеяния показывает связь между двумя числовыми признаками; каждая точка соответствует одному пациенту.

PCA (Principal Component Analysis, метод главных компонент) - линейный метод понижения размерности. Он проецирует данные в пространство меньшей размерности, сохраняя как можно большую долю дисперсии [6]. В работе PCA используется для отображения многомерной матрицы признаков на плоскости.

t-SNE (t-Distributed Stochastic Neighbor Embedding) - нелинейный метод визуализации многомерных данных. Он старается сохранить локальную близость объектов: если пациенты были похожи в исходном пространстве признаков, они должны оказаться рядом на двумерном графике [7]. t-SNE полезен для поиска локальных групп и областей смешения классов.

1.6. Метрики качества классификации

Хотя основная цель работы - предобработка, в практической части выполняется проверка подготовленных данных на логистической регрессии. Для оценки используются стандартные метрики `classification_report` из `scikit-learn` [8].

Accuracy - доля правильных ответов среди всех объектов:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Precision показывает, какая доля объектов, предсказанных как положительные, действительно является положительной:

$$\text{Precision} = TP / (TP + FP)$$

Recall показывает, какую долю реальных положительных объектов модель нашла:

$$\text{Recall} = TP / (TP + FN)$$

F1-score - гармоническое среднее precision и recall:

$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

Здесь TP - верно предсказанные положительные объекты, TN - верно предсказанные отрицательные объекты, FP - ложноположительные ошибки, FN - ложноотрицательные ошибки.

2. Практическая часть

2.1. Основные компоненты программной реализации

Основной код реализации приведён в приложении А. В таблице ниже указаны программные сущности, которые реализуют интеллектуальную

часть работы.

Компонент	Назначение	Источник
<code>pandas.read_csv</code>	Загрузка processed-файлов UCI Heart Disease и преобразование символа ? в NaN	Библиотека pandas
<code>drop_duplicates</code>	Удаление полных дубликатов	Библиотека pandas
<code>SimpleImputer</code>	Заполнение пропусков медианой и модой	scikit-learn [3]
<code>StandardScaler</code>	Стандартизация числовых признаков	scikit-learn [4]
<code>OneHotEncoder</code>	Кодирование категориальных признаков	scikit-learn [5]
<code>ColumnTransformer</code>	Единый конвейер обработки разных типов признаков	scikit-learn
<code>make_statistical_augmentation</code>	Авторская функция статистической аугментации табличных данных	Реализовано в работе
<code>PCA</code>	Линейное понижение размерности для визуализации	scikit-learn [6]
<code>TSNE</code>	Нелинейная визуализация локальной структуры данных	scikit-learn [7]
<code>LogisticRegression</code>	Проверочная модель бинарной классификации	scikit-learn

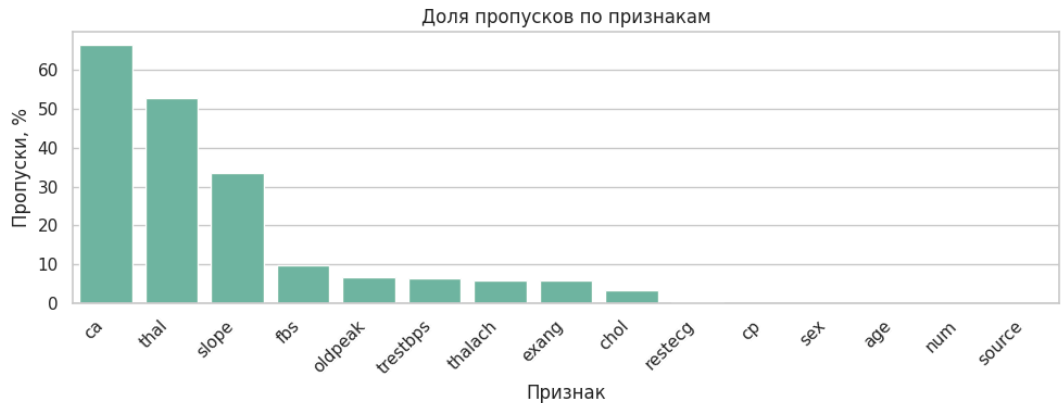
2.2. Загрузка и первичный анализ данных

В практической части были объединены четыре processed-файла. После объединения набор содержал 920 строк и 15 колонок, включая добавленную колонку source, указывающую источник записи.

Источник	Количество строк
Cleveland	303
Hungarian	294
Long Beach VA	200
Switzerland	123

На этапе анализа качества данных были рассчитаны пропуски по признакам. Наибольшая доля пропусков обнаружена в признаках ca и thal.

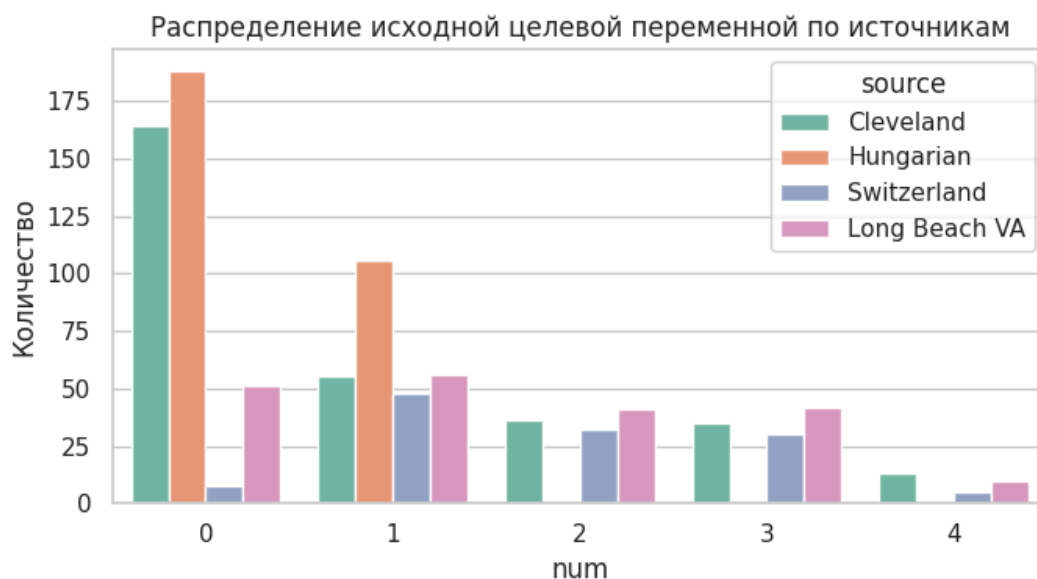
Признак	Количество пропусков	Доля пропусков, %
ca	611	66.41
thal	486	52.83
slope	309	33.59
fbs	90	9.78
oldpeak	62	6.74
trestbps	59	6.41
thalach	55	5.98
exang	55	5.98
chol	30	3.26
restecg	2	0.22



Также было найдено 2 полных дубликата. При проверке подозрительных значений выявлено 172 нулевых значения chol и 1 нулевое значение

trestbps. Для этих медицинских признаков ноль является физически неправдоподобным значением, поэтому такие значения были обработаны как скрытые пропуски.

Исходная целевая переменная num распределена следующим образом: 411 объектов имеют значение 0, 265 - значение 1, 109 - значение 2, 107 - значение 3 и 28 - значение 4.



2.3. Реализованная предобработка

Предобработка выполнялась в следующей последовательности: удаление дубликатов, бинаризация целевой переменной, обработка скрытых пропусков, удаление признака са, создание индикаторов пропусков, импутация, стандартизация и one-hot encoding.

Признак са был удалён, так как доля пропусков после обработки скрытых пропусков превысила 60%. Признак thal был сохранён, несмотря на большую долю пропусков, поскольку он является диагностически значимым категориальным признаком и не превысил выбранный порог удаления.

Для признаков с пропусками были добавлены индикаторы вида trestbps_was_missing, chol_was_missing, fbs_was_missing, restecg_was_missing, thalach_was_missing, exang_was_missing, oldpeak_was_missing, slope_was_missing и thal_was_missing. Эти признаки сохраняют информацию о том, что значение было восстановлено, а не наблюдалось напрямую.

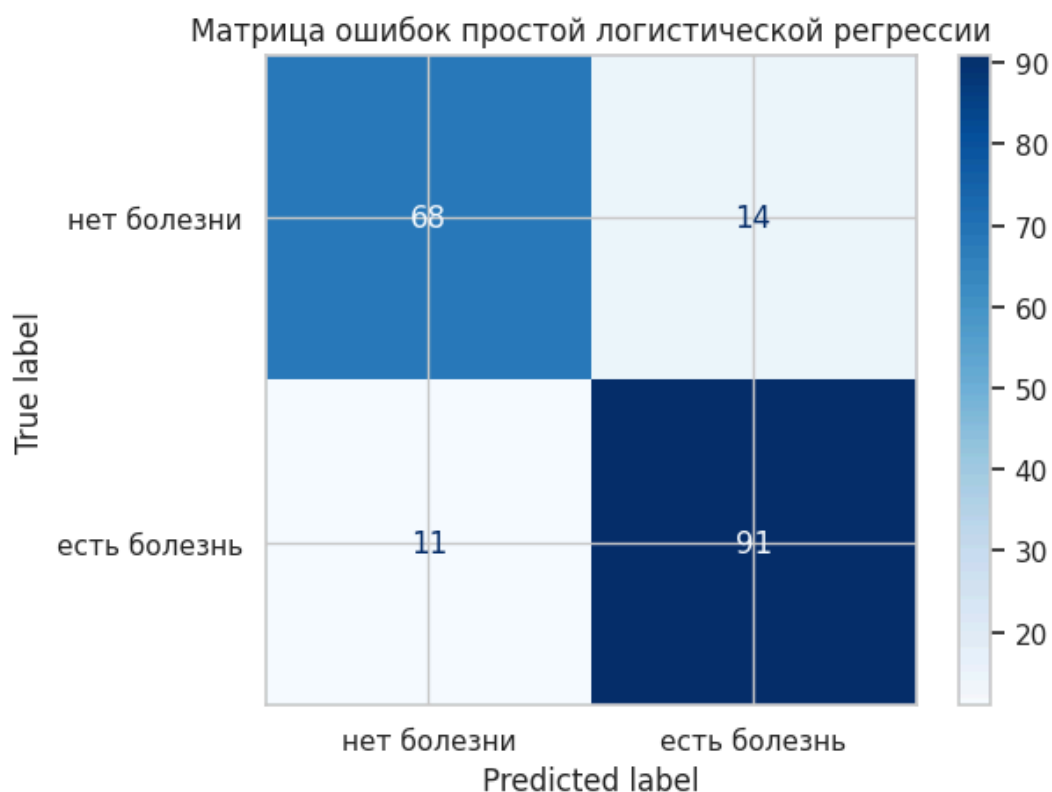
После предобработки итоговая матрица признаков X_prepared содержит 918 строк и 33 признака. С учётом служебных колонок target, severity_num и source сохранённый файл heart_disease_preprocessed.csv имеет размер 918 строк и 36 колонок.

Показатель	Значение
Строк после удаления дубликатов	918
Класс 0 после бинаризации	410
Класс 1 после бинаризации	508
Признак, удалённый по доле пропусков	ca
Количество признаков после one-hot encoding	33
Итоговый файл	heart_disease_preprocessed.csv

2.4. Проверка пригодности данных на модели

Для проверки того, что предобработанные данные пригодны для машинного обучения, была обучена логистическая регрессия. Данные были разделены на обучающую и тестовую выборки в отношении 80% к 20%, с сохранением пропорций классов через stratify=y.

Класс	Precision	Recall	F1-score	Support
Нет болезни	0.86	0.83	0.84	82
Есть болезнь	0.87	0.89	0.88	102
Accuracy			0.86	184



Матрица ошибок показывает, что из 82 объектов класса «нет болезни» модель правильно определила 68, а 14 ошибочно отнесла к классу «есть болезнь». Из 102 объектов класса «есть болезнь» модель правильно определила 91, а 11 ошибочно отнесла к классу «нет болезни». Это подтверждает, что после предобработки данные имеют корректный формат и содержат полезный классификационный сигнал.

2.5. Аугментация данных

Для демонстрации аугментации была реализована функция `make_statistical_augmentation`. Она балансирует классы: выбирает меньший класс, сэмплирует его строки с возвращением и добавляет небольшой гауссов шум к числовым признакам. Категориальные признаки сохраняются без изменений.

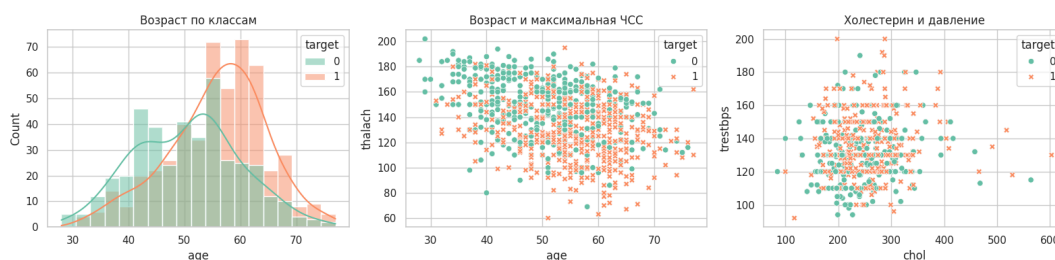
В исходных предобработанных данных меньшим классом является класс 0: 410 объектов против 508 объектов класса 1. Поэтому было создано 98 дополнительных объектов класса 0. После аугментации оба класса содержат по 508 объектов.

target	Исходные объекты	Аугментированные объекты	Итого
0	410	98	508
1	508	0	508

Итоговый аугментированный набор сохранён в файл `heart_disease_augmented.csv`. Его размер составляет 1016 строк и 14 колонок. Такой подход не заменяет сбор реальных медицинских данных, однако показывает корректный учебный принцип создания статистически близких табличных объектов.

2.6. Визуализация данных

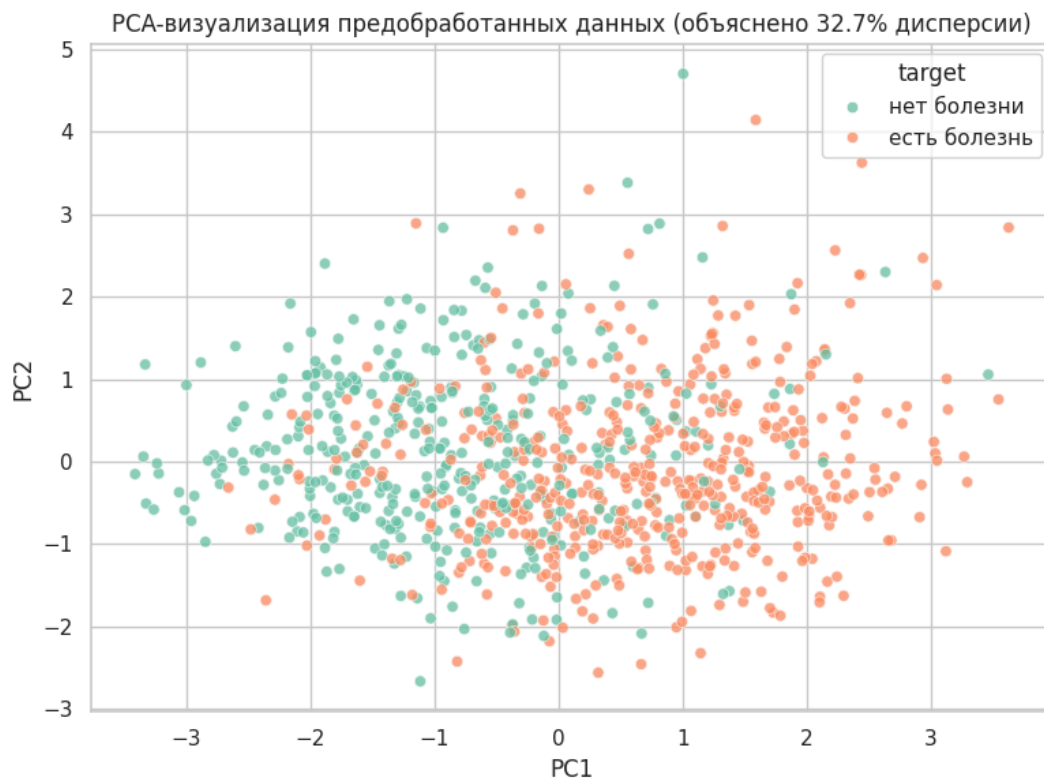
В работе построены обычные графики по отдельным признакам и графики с понижением размерности.



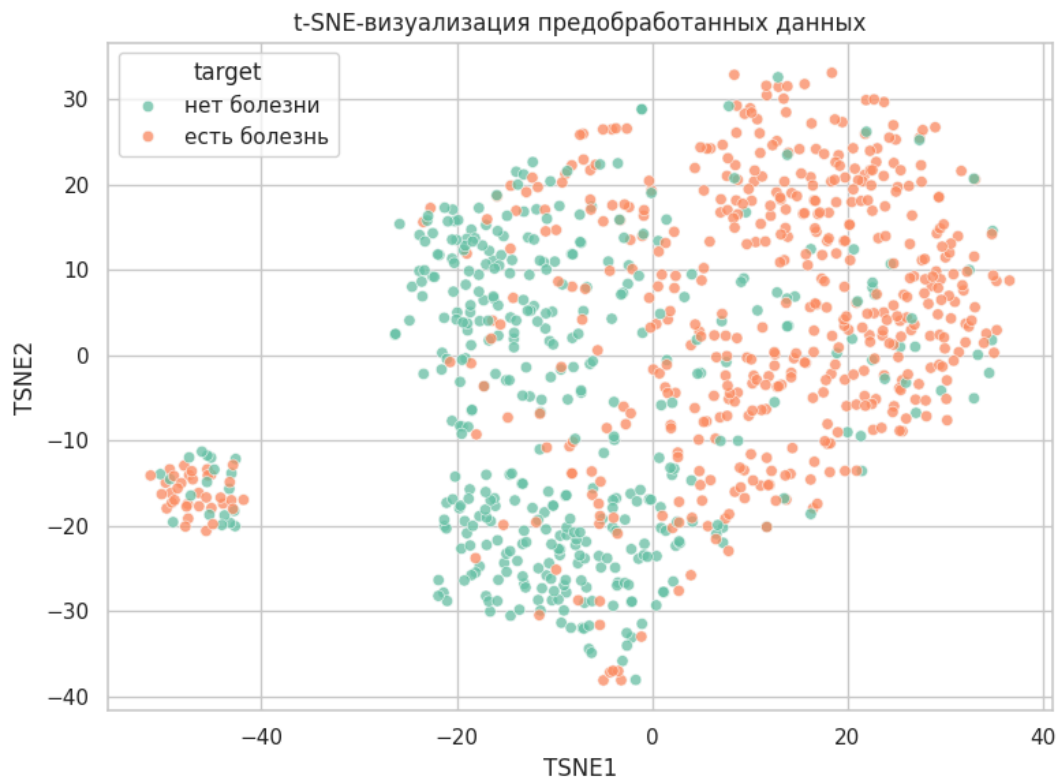
Первый график на рисунке 4 - гистограмма возраста по классам. Она показывает, как распределены пациенты разных классов по возрасту. Если распределения классов перекрываются, возраст сам по себе не является достаточным признаком для точной классификации, но может быть полезен в сочетании с другими признаками.

Второй график - диаграмма рассеяния `age` и `thalach`. Он показывает связь возраста и максимальной частоты сердечных сокращений. Каждая точка соответствует пациенту, цвет показывает класс. Такой график позволяет оценить, есть ли области, где один класс встречается чаще другого.

Третий график - диаграмма рассеяния `chol` и `trestbps`. Он показывает совместное поведение уровня холестерина и давления в покое. Эти признаки имеют медицинский смысл как факторы риска, поэтому их совместная визуализация помогает оценить возможные области повышенного риска и наличие выбросов.



На рисунке 5 показана PCA-проекция данных на две главные компоненты. Каждая точка соответствует пациенту, а оси PC1 и PC2 являются новыми синтетическими координатами, построенными из исходных признаков. График позволяет оценить, существует ли линейная разделимость классов в низкоразмерном представлении.



На рисунке 6 показана t-SNE-визуализация. В отличие от PCA, t-SNE является нелинейным методом и лучше отражает локальную близость объектов. Если на графике видны локальные группы, это означает, что в исходном пространстве признаков существуют группы похожих пациентов. При этом t-SNE следует использовать именно как инструмент качественной визуальной интерпретации, а не как строгую численную метрику расстояний между классами.

Заключение

В ходе практической работы был выполнен полный цикл предобработки медицинского табличного набора данных UCI Heart Disease для задачи бинарной классификации наличия сердечного заболевания. Были объединены данные из четырёх processed-файлов, выполнен анализ качества данных, обнаружены пропуски, скрытые пропуски, дубликаты и особенности распределения целевой переменной.

В процессе предобработки были удалены полные дубликаты, целевая переменная num была преобразована в бинарный target, скрытые пропуски в chol и trestbps были заменены на NaN, признак ca был удалён из-за чрезмерной доли пропусков. Для остальных признаков были выполнены импутация, стандартизация числовых признаков и one-hot encoding категориальных признаков. Также были добавлены индикаторы пропусков.

В результате был получен файл heart_disease_preprocessed.csv размером 918 строк и 36 колонок. Проверка на логистической регрессии показала ассигасу 0.86, что подтверждает пригодность подготовленных данных для последующего обучения интеллектуальных моделей. Дополнительно была выполнена статистическая аугментация меньшего класса, в результате чего размер набора увеличился до 1016 строк, а классы были сбалансированы по 508 объектов.

Также были построены визуализации отдельных признаков, PCA и t-SNE. Они позволили оценить распределение медицинских признаков, пересечение классов и структуру данных в двумерных проекциях. Таким образом, цель практической работы достигнута: данные были подготовлены, сохранены и проверены на пригодность для дальнейшего интеллектуального анализа.

Список использованных источников

1. World Health Organization. Cardiovascular diseases (CVDs). URL: [https://www.who.int/en/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/en/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds))

2. UCI Machine Learning Repository. Heart Disease Dataset. URL: <https://archive.ics.uci.edu/dataset/45/heart%2Bdisease>
3. scikit-learn documentation. SimpleImputer. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>
4. scikit-learn documentation. StandardScaler. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
5. scikit-learn documentation. OneHotEncoder. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>
6. scikit-learn documentation. PCA. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>
7. scikit-learn documentation. TSNE. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>
8. scikit-learn documentation. classification_report. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

Приложение А

Основной программный код:


```

from pathlib import Path
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.manifold import TSNE
from sklearn.metrics import ConfusionMatrixDisplay, classification_report
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid", palette="Set2")

DATA_DIR = Path("heart+disease")

columns = [
    "age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
    "thalach", "exang", "oldpeak", "slope", "ca", "thal", "num"
]

source_files = {
    "Cleveland": "processed.cleveland.data",
    "Hungarian": "processed.hungarian.data",
    "Switzerland": "processed.switzerland.data",
    "Long Beach VA": "processed.va.data",
}

frames = []
for source, filename in source_files.items():
    part = pd.read_csv(
        DATA_DIR / filename,
        header=None,
        names=columns,
        na_values="?",
    )
    part["source"] = source
    frames.append(part)

raw = pd.concat(frames, ignore_index=True)

df = raw.copy()
df = df.drop_duplicates().reset_index(drop=True)
df["target"] = (df["num"] > 0).astype(int)

```

```

df.loc[df["chol"] == 0, "chol"] = np.nan
df.loc[df["trestbps"] == 0, "trestbps"] = np.nan

initial_features = [
    "age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
    "thalach", "exang", "oldpeak", "slope", "ca", "thal"
]

missing_rate_after_zero_fix = df[initial_features].isna().mean
high_missing_cols = missing_rate_after_zero_fix[
    missing_rate_after_zero_fix > 0.60
].index.tolist()

model_features = [col for col in initial_features if col not in
cols_with_missing]
cols_with_missing = [col for col in model_features if df[col].

for col in cols_with_missing:
    df[f"{col}_was_missing"] = df[col].isna().astype(int)

indicator_features = [f"{col}_was_missing" for col in cols_wit

numeric_features = ["age", "trestbps", "chol", "thalach", "olc
categorical_features = ["sex", "cp", "fbs", "restecg", "exang"

X = df[model_features + indicator_features]
y = df["target"]

numeric_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

categorical_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", spar
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
        ("missing_flag", "passthrough", indicator_features),
    ],
    remainder="drop",
    verbose_feature_names_out=False,
)

preprocessor.set_output(transform="pandas")

```

```

preprocessor.set_output(transform="pandas")
X_prepared = preprocessor.fit_transform(X)

prepared = X_prepared.copy()
prepared["target"] = y.to_numpy()
prepared["severity_num"] = df["num"].to_numpy()
prepared["source"] = df["source"].to_numpy()
prepared.to_csv("heart_disease_preprocessed.csv", index=False)

X_train, X_test, y_train, y_test = train_test_split(
    X_prepared,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)

model = LogisticRegression(max_iter=2000, class_weight="balanced")
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred, target_names=["Heart Disease", "No Heart Disease"]))

def make_statistical_augmentation(data, target_col, numeric_cols, categorical_cols, random_state):
    rng = np.random.default_rng(random_state)
    base = data[numeric_cols + categorical_cols + [target_col]]

    for col in numeric_cols:
        base[col] = base[col].fillna(base[col].median())
    for col in categorical_cols:
        base[col] = base[col].fillna(base[col].mode(dropna=True)[0])

    counts = base[target_col].value_counts()
    majority_size = counts.max()
    augmented_parts = []

    for cls, count in counts.items():
        need = int(majority_size - count)
        if need <= 0:
            continue

        cls_rows = base[base[target_col] == cls]
        sampled = cls_rows.sample(n=need, replace=True, random_state=rng)

        for col in numeric_cols:
            std = cls_rows[col].std()
            noise_scale = 0.03 * std if pd.notna(std) and std > 0 else 0
            sampled[col] = sampled[col] + rng.normal(0, noise_scale)

            low, high = base[col].quantile([0.01, 0.99])
            sampled[col] = sampled[col].clip(low, high)

        sampled["is_augmented"] = 1
    return base.append(sampled)

```

```

        augmented_parts.append(sampled)

original = base.copy()
original["is_augmented"] = 0

if augmented_parts:
    return pd.concat([original] + augmented_parts, ignore_index=True)
return original

augmented = make_statistical_augmentation(
    df,
    target_col="target",
    numeric_cols=numeric_features,
    categorical_cols=categorical_features,
    random_state=42,
)

augmented.to_csv("heart_disease_augmented.csv", index=False)

pca = PCA(n_components=2, random_state=42)
pca_coords = pca.fit_transform(X_prepared)

pca_for_tsne = PCA(n_components=min(30, X_prepared.shape[1]),
    random_state=42)
tsne_input = pca_for_tsne.fit_transform(X_prepared)

tsne = TSNE(
    n_components=2,
    perplexity=30,
    init="pca",
    learning_rate="auto",
    max_iter=1000,
    random_state=42,
)
tsne_coords = tsne.fit_transform(tsne_input)

```